AKADEMIA NAUK STOSOWANYCH
W NOWYM SĄCZU

Wydział Nauk Inżynieryjnych
Katedra Informatyki

DOKUMENTACJA PROJEKTOWA

ZAAWANSOWANE PROGRAMOWANIE

...Algorytm Binary Search Tree z zastosowaniem GitHub...

Autor:
Kacper Twaróg
Kszysztof Zdzieszyński

Prowadzący:
mgr inż. Dawid Kotlarski

Nowy Sącz 2026

Spis treści

1. Ogólne określenie wymagań **3**

2. Analiza problemu **9**

2.1. Podstawy matematyczne i analiza algorytmiczna BST 11

2.2. Złożoność czasowa i pamięciowa 11

2.3. Problem z zapisem wskaźników do pliku 12

2.4. Zapis tekstowy a prawdziwa serializacja binarna 12

2.5. Zarządzanie pamięcią i problem przepełnienia stosu 13

3. Projektowanie **15**

3.1. Narzędzia, technologie i ustawienia kompilatora 15

3.2. Architektura systemu i diagram klas 15

3.3. Zarządzanie kodem (Git) i generowanie dokumentacji 19

4. Implementacja **21**

4.1. Szczegóły implementacyjne i wybrane fragmenty kodu 21

4.2. Powstałe wyniki działania programu 25

5. Wnioski **27**

Literatura **28**

Spis rysunków **28**

Spis tabel **29**

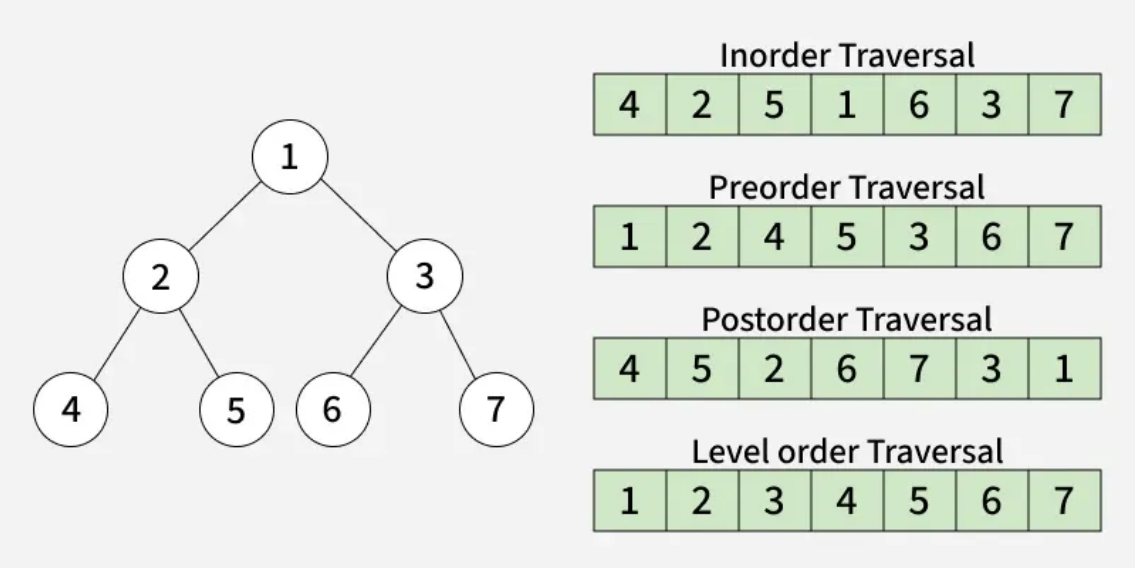
Spis listingów **30**

1. Ogólne określenie wymagań

Podstawą programu jest binarne drzewo wyszukiwania (BST), którego węzły są tworzone dynamicznie. Każdy węzeł przechowuje jedną wartość typu `int` oraz wskaźniki na lewe i prawe dziecko. Wartości mniejsze od wartości danego węzła są umieszczane w jego lewym poddrzewie, a większe — w prawym. Wstawianie i usuwanie elementów odbywa się rekurencyjnie. Próba dodania wartości, która już znajduje się w drzewie, nie powoduje utworzenia kolejnego węzła.

Aby wizualizować hierarchiczną strukturę i poruszać się po niej, program implementuje trzy główne metody prezentowania binarnego drzewa wyszukiwania: preorder, inorder i postorder, z których każda określa kolejność wyświetlania lewego poddrzewa, korzenia i prawego poddrzewa.

Preorder odwiedza najpierw korzeń, następnie lewe poddrzewo i prawe poddrzewo. Postorder odwiedza lewe poddrzewo, prawe poddrzewo, a na końcu korzeń. W typowym przejściu inorder odwiedza się lewe poddrzewo, korzeń i prawe poddrzewo. W tej implementacji funkcja nazwana `InorderTraversalNode` wykonuje jednak kolejność odwrotną: prawe poddrzewo, korzeń, lewe poddrzewo. W efekcie wypisuje wartości malejąco. Wszystkie trzy metody są przejściami w głąb (DFS). Przejście wszerz (BFS), odwiedzające węzły poziomami, nie zostało zaimplementowane.



Rys. 1.1. Binary Search Tree Wizualny Pokaz

Menedżer plików zapisuje wartości z drzewa do pliku binarnego i umożliwia ich późniejsze wczytanie. Użytkownik podaje nazwę pliku, a podczas wczytywania może

wybrać zastąpienie aktualnego drzewa albo dodanie odczytanych wartości do istniejącego drzewa. Zapis odbywa się w kolejności preorder: do pliku trafiają kolejne wartości typu `int`, a nie całe obiekty węzłów ani ich wskaźniki. Wykorzystywane są do tego `std::ios::binary`, `file.write()`, `file.read()` oraz `reinterpret_cast`. Program jest obsługiwany z poziomu menu tekstowego działającego w pętli `while` i instrukcji `switch`.

Menu programu udostępnia dodawanie i usuwanie pojedynczych wartości, usuwanie całego drzewa, trzy sposoby wyświetlania jego struktury, wyszukiwanie wartości wraz z prezentacją ścieżki, zapis do pliku oraz dwa tryby wczytywania danych: zastąpienie aktualnego drzewa lub dodanie wartości do niego. Dostępna jest również opcja utworzenia przykładowego zestawu danych i zakończenia programu. Wstawianie odbywa się rekurencyjnie, a ponowne wstawienie tej samej wartości nie powoduje utworzenia duplikatu.

Github

GitHub służy jako oparta na chmurze platforma do kontroli wersji Git, zapewniając uporządkowane środowisko do śledzenia rozwoju projektu i zarządzania jego historią, a także pomaga studentom (nam) nauczyć się pracy zespołowej. Dzięki systemowi pull requestów, wypychania i pobierania zmian oraz gałęzi umożliwia zespołowi pracę nad projektem za pomocą łatwego w obsłudze interfejsu, co sprawia, że współpraca jest bezproblemowa. Jest bardzo popularny w internecie i wykorzystywany przez wiele aplikacji ze względu na swoją pomysłowość: automatyzuje i przyspiesza modyfikowanie bazy kodu, zapewniając przy tym wysoki poziom bezpieczeństwa, który dzięki zabezpieczeniom i migawkom stanu projektu zapobiega złośliwym działaniom i głupim błędom. Podczas tworzenia drzewa BST i systemu zarządzania plikami GitHub pełni funkcję zdalnego repozytorium, w którym wszystkie wersje kodu źródłowego są bezpiecznie archiwizowane i monitorowane na przestrzeni czasu.

Przepływ pracy wykorzystuje gałęzie do pracy nad poszczególnymi aspektami projektu bez obaw, że zepsuje się go całkowicie. Dzięki odizolowaniu wprowadzonych zmian łatwo ustalić, co zostało zepsute, jeśli pracuje się tylko na określonej gałęzi, podczas gdy w normalnych warunkach nie byłoby wiadomo na pewno, co i dlaczego przestało działać. Umożliwia to debugowanie bez zbędnych komplikacji, a narzędzia udostępniane przez IDE upraszczają pracę programisty, pozwalając mu skupić się na najważniejszych aspektach naprawiania kodu: znalezieniu przyczyny problemu lub sprawieniu, by kod działał. Operacje scalania i commity zapewniają historyczny zapis ewolucji projektu oraz bezpośredni, niepodważalny dowód jego rozwoju. Co więcej, GitHub wymusza rygorystyczną higienę kodu poprzez strategiczne wykorzystanie konfiguracji .gitignore, dzięki czemu skompilowane pliki binarne (.exe, pliki obiektowe), lokalne katalogi IDE i tymczasowe artefakty systemowe są wykluczane z kontroli wersji.

CLion - IDE Projektu

CLion pełni funkcję głównego środowiska IDE w tym projekcie, oferując specjalnie zaprojektowaną przestrzeń roboczą do pracy z C i C++. Dzięki systemowi zarządzania kompilacją CMake CLion automatyzuje proces kompilacji, zarządza zależnościami nagłówków i przeprowadza analizę statyczną kodu w czasie rzeczywistym, aby z łatwością wykrywać błędy składni. Dzięki temu jest bardzo przydatny, oferując funkcje niedostępne w Visual Studio ani Visual Studio Code. Jego główną wadą jest oczywiście to, że jest dostępny w ramach subskrypcji, co przekłada się na zauważalnie bogaty zestaw funkcji.

IDE znacznie przyspiesza debugowanie dzięki wbudowanym graficznym narzędziom do inspekcji. Podobnie jak w przypadku innych narzędzi, można ustawiać warunkowe punkty przerwania, analizować bieżące ramki oraz śledzić wskaźniki i pamięć dynamiczną w miarę działania programu. Ta funkcja okazuje się kluczowa podczas rozwiązywania problemów związanych ze złożonymi operacjami na pamięci lub nietypowym zachowaniem, które trudno wykryć w prosty sposób, i jest jednym z powodów, dla których CLion sprawdza się tak dobrze — wychwytuje wiele drobnych problemów, które w przeciwnym razie mogłyby szybko przerodzić się w poważne usterki.

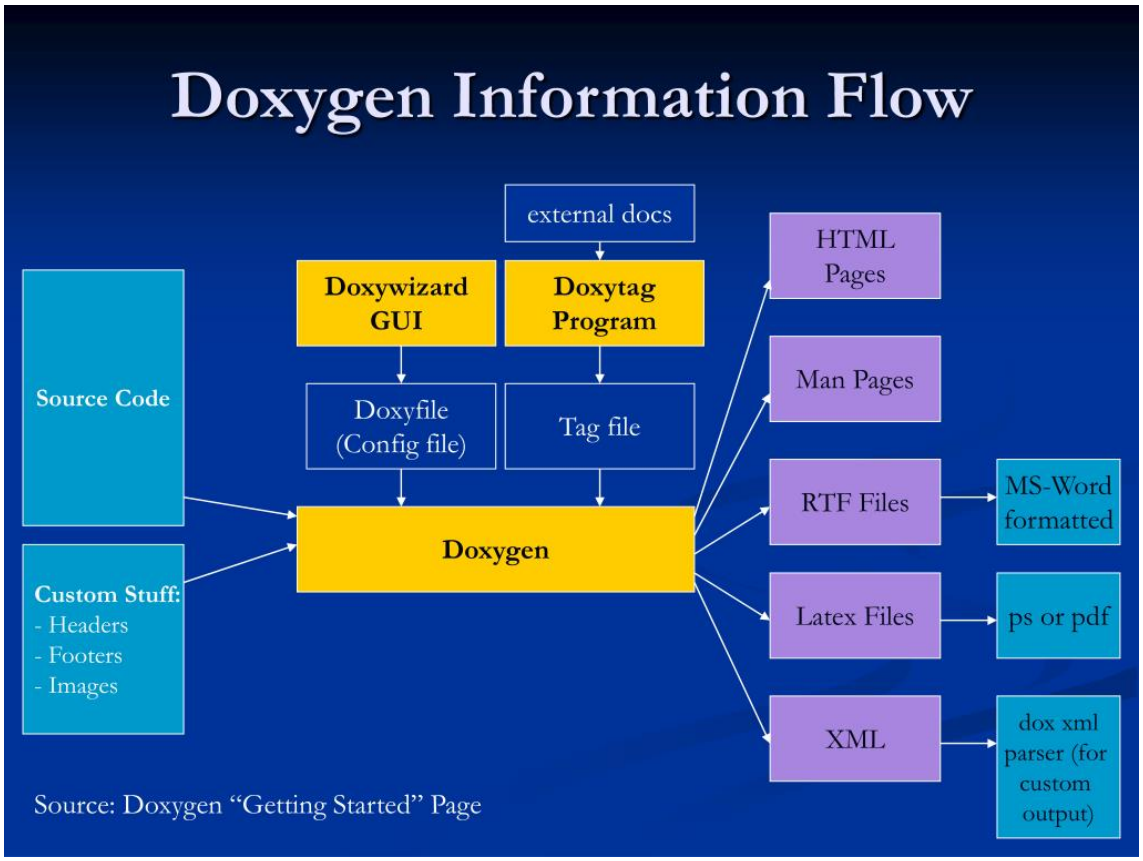
Doxygen + Overleaf

Doxygen to automatyczny generator dokumentacji dostosowany do języków programowania, takich jak w naszym przypadku C++, który pozwala nam generować dokumentację do wykorzystania w przyszłości, jeśli użyjemy specjalnych bloków formatujących do opisu naszego kodu. To rozwinięcie zwykłego komentowania kodu, które umożliwia tworzenie specjalnie przygotowanej dokumentacji, która — jeśli jest aktualizowana — doskonale wyjaśnia działanie funkcji. Analizując bloki komentarzy zawierające odpowiednie znaczniki (@brief, @param, @return) bezpośrednio w plikach nagłówkowych i implementacjach źródłowych, Doxygen wyodrębnia hierarchie klas i zmienne, aby po zbudowaniu utworzyć wizualną dokumentację. Wynik jest generowany w formacie LaTeX za pomocą Doxywizard, narzędzia instalowanego wraz z Doxygen. Pozwala nam ono dostosować sposób, w jaki Doxygen generuje dane w formacie LaTeX / PDF / innym, określić, co i jak wyświetla, a także dostosować widoczność.

Z drugiej strony Overleaf to oparta na chmurze, współpracująca platforma do edycji LaTeX-a, która umożliwia łatwe tworzenie profesjonalnie wyglądającej dokumentacji za pomocą kodu. W porównaniu z tradycyjnymi sposobami pracy z LaTeX-em, które ze względu na liczbę modułów i elementów wymagałyby wielu instalacji i dystrybucji, Overleaf oferuje nie tylko edytor online, który nie wymaga instalowania czegokolwiek na komputerze, lecz także możliwość współpracy z innymi w czasie rzeczywistym, pozwalając dwóm lub większej liczbie osób jednocześnie edytować dokument i widzieć nawzajem wprowadzane zmiany, podobnie jak w Dokumentach Google; eliminuje też problem brakujących zależności, które nie są domyślnie dostępne dla używanych funkcji (nie dotyczy to sytuacji, gdy dodajesz własną zawartość, która nie jest domyślnie dostępna w dokumencie Overleaf), zapewnia zdalną kompilację, która w bezpłatnych planach ma limit, ale jest szybsza niż kompilacja lokalna, a ogólnie rzecz biorąc, stanowi nowoczesną implementację LaTeX-a, oferującą większą elastyczność dzięki podejściu online.

Jeśli chodzi o Doxygen do dokumentacji, powodów, dla których go tutaj używamy, jest kilka: przy odpowiedniej konfiguracji i konsekwentnym programiście aktualizowane są zarówno funkcje, jak i dokumentacja, dzięki czemu każdy wynik jest automatycznie aktualizowany bez żadnego wysiłku — oczywiście, jeśli programista jest leniwy i nie zaktualizuje komentarzy, dokumentacja pozostanie nieprawidłowa. Narzędzie to eliminuje jednak żmudne, powtarzalne czynności związane z pisaniem lub edytowaniem dokumentacji, które mogą szybko dać się komuś we znaki. W niektó-

rych przypadkach automatycznie tworzy diagramy dziedziczenia i grafy współpracy, których ręczne opracowanie w zewnętrznym programie do tworzenia diagramów zajęłoby wiele godzin. Ogólnie rzecz biorąc, choć nie jest doskonałym narzędziem i brakuje mu ludzkiej świadomości, jest bardzo przydatne do dokumentowania kodu projektu i stanowi drugi rodzaj dokumentacji, który będziemy udostępniać w ramach tego projektu.

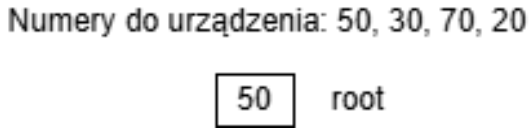


Rys. 1.2. Doxygen diagram jak dziala

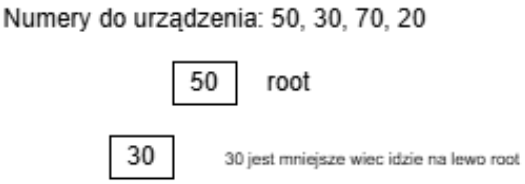
2. Analiza problemu

Drzewo binarnych wyszukiwań (BST) jest strukturą danych służącą do przechowywania wartości w uporządkowany sposób. Może być wykorzystywane między innymi w implementacjach słowników, zbiorów oraz struktur, w których potrzebne jest wyszukiwanie, wstawianie i usuwanie elementów. W dobrze zrównoważonym drzewie operacje te mogą działać w czasie logarytmicznym, jednak zwykłe BST nie utrzymuje automatycznie równowagi. W przypadku niekorzystnej kolejności wstawiania może powstać drzewo skośne, a wtedy podstawowe operacje mają złożoność liniową. Każdy węzeł zawiera wartość oraz wskaźniki na lewe i prawe poddrzewo. Wartości mniejsze od wartości węzła znajdują się po lewej stronie, a większe — po prawej. Korzeń jest pierwszym węzłem drzewa, od którego rozpoczyna się wyszukiwanie i pozostałe operacje.

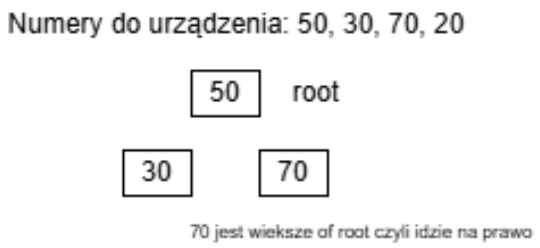
Aby praktycznie zrozumieć te operacje, rozważmy ręczny przykład krok po kroku, polegający na wstawieniu sekwencji pięćdziesiąt, trzydzieści, siedemdziesiąt i dwadzieścia do początkowo pustego drzewa.



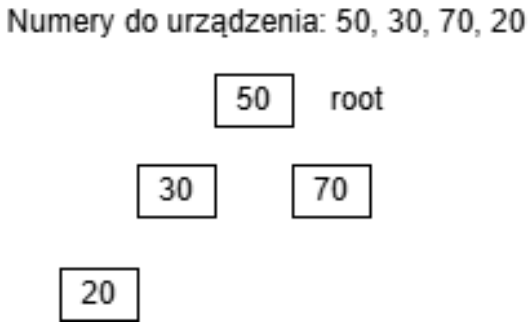
Wstawienie pięćdziesiąt tworzy węzeł główny o wartości pięćdziesiąt i pustych dzieciach. Następnie wstawienie trzydzieści porównuje trzydzieści z pięćdziesiąt; ponieważ trzydzieści jest mniejsze, a lewe dziecko jest puste, nowy węzeł zawierający trzydzieści zostaje utworzony jako lewe dziecko pięćdziesiąt.



Wstawienie siedemdziesiąt porównuje siedemdziesiąt z pięćdziesiąt; ponieważ siedemdziesiąt jest większe, a prawe dziecko jest puste, nowy węzeł zawierający siedemdziesiąt zostaje utworzony jako prawe dziecko pięćdziesiąt.



Na koniec wstawienie dwadzieścia porównuje dwadzieścia z pięćdziesiąt, przechodzi do lewego dziecka trzydzieści, a ponieważ dwadzieścia jest mniejsze od trzydzieści, a jego lewe dziecko jest puste, dwadzieścia zostaje umieszczone jako lewe dziecko trzydzieści.



Jeśli następnie wyszukiwalibyśmy dwadzieścia przy użyciu logiki znajdowania ścieżki, algorytm sprawdziłby pięćdziesiąt, przeszedł w lewo do trzydzieści, przeszedł w lewo do dwadzieścia, pomyślnie dopasowałby wartość docelową i zapisałby ścieżkę przejścia. To prosty opis słowny tego, jak przykład mógłby wyglądać przedstawiony słowami; zostanie on zrealizowany za pomocą dynamicznych struktur obsługujących surową pamięć w najbardziej zoptymalizowany sposób, jaki możemy zastosować: poprzez dynamiczną alokację pamięci, dzięki czemu nie potrzeba więcej pamięci niż na przechowywaną liczbę całkowitą.

Jak dotąd omówiliśmy drzewa wyszukiwań binarnych w bardzo podstawowy sposób, a aby w pełni zrozumieć, jak działają, musimy omówić ich złożoność czasową oraz to, jak działają w teorii i praktyce, a także jakie są ich realistyczne zastosowania w prawdziwym świecie.

Drzewa wyszukiwań binarnych w szczegółach

Drzewa BST są przykładem struktury, która może być używana do implementacji słowników i zbiorów oraz do przechowywania danych wymagających uporządkowa-

nia. Podobną rolę w niektórych zastosowaniach pełnią także inne struktury drzewiaste. Warto odróżnić BST od drzew B i B+: nie są one rozszerzeniami BST, lecz osobnymi strukturami, często wykorzystywanymi w indeksach baz danych. Drzewa wyszukiwań mogą być również stosowane w wybranych rozwiązaniach związanych z tablicami symboli, jednak konkretna struktura zależy od wymagań danego kompilatora lub systemu. Nie należy zakładać, że każdy system bazodanowy, plikowy lub routingu wykorzystuje zwykłe drzewo BST.

Aby formalnie analizować drzewo binarnych wyszukiwań, informatycy wykorzystują matematyczne równania rekurencyjne, teorię zbiorów i kombinatorykę, by dowodzić efektywności czasowej i pamięciowej.

2.1. Podstawy matematyczne i analiza algorytmiczna BST

Wysokość drzewa (h) definiowana jest jako maksymalna liczba krawędzi od korzenia do liścia. Dla drzewa idealnie zrównoważonego o N węzłach, maksymalna liczba węzłów wynosi:

$$N_{\max} = \sum_{i=0}^{h} 2^i = 2^{h+1} - 1 \tag{2.1}$$

Minimalna wysokość takiego drzewa wyraża się wzorem logarytmicznym:

$$h_{\min} = \lfloor \log_2(N) \rfloor \tag{2.2}$$

W najgorszym scenariuszu, gdy drzewo degeneruje się do listy liniowej (drzewo skośne), wysokość osiąga wartość:

$$h_{\max} = N - 1 \tag{2.3}$$

2.2. Złożoność czasowa i pamięciowa

Efektywność algorytmiczna operacji na drzewie BST zależy bezpośrednio od jego wysokości. Stosując relacje rekurencyjne typu $T(N) = T(N/2) + \mathcal{O}(1)$, otrzymujemy następujące klasy złożoności:

Pamięć zajmowana przez węzły wynosi $\mathcal{O}(N)$ dla N elementów. Dodatkowo rekurencyjne operacje wykorzystują stos wywołań o rozmiarze $\mathcal{O}(h)$, gdzie h oznacza wysokość drzewa. W przypadku drzewa silnie niezrównoważonego wysokość może być rzędu N, co zwiększa zużycie stosu i może prowadzić do jego przepełnienia.

Zanim przejdziemy dalej, powinniśmy omówić kilka szczegółów dotyczących drzew

Operacja	Drzewo zrównoważone	Drzewo skośne (najgorszy przypadek)
Wyszukiwanie	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
Wstawianie	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
Usuwanie	$\mathcal{O}(\log N)$	$\mathcal{O}(N)$
Przechodzenie	$\mathcal{O}(N)$	$\mathcal{O}(N)$

Tab. 2.1. Złożoność operacji na drzewie BST.

binarnych wyszukiwań: konkretnie, dlaczego zapis do plików może być trudny, dlaczego w tym projekcie używamy prawdziwego formatu binarnego zamiast tekstu do ich zapisywania, a także zarządzanie pamięcią i unikanie przepełnienia stosu. Trzy banały, jeśli można tak powiedzieć.

2.3. Problem z zapisem wskaźników do pliku

Zapisanie drzewa BST do pliku to wbrew pozorom dość problematyczna kwestia. W pamięci RAM nasze drzewo to po prostu zbiór dynamicznie utworzonych węzłów połączonych wskaźnikami, co pozwala łatwo po nich skakać. Problem polega na tym, że na dysku twardym dane muszą być zapisane liniowo (jeden po drugim), a same adresy wskaźników po wyłączeniu programu tracą jakąkolwiek ważność.

Najważniejsze to nie popsuć przy tym kształtu drzewa. Jeśli zapiszemy wartości w kolejności inorder, otrzymamy ciąg uporządkowany. Ponowne wstawianie takiego ciągu do pustego BST może utworzyć drzewo skośne, w którym każdy kolejny element trafia do prawego poddrzewa. Zapis w kolejności preorder pozwala odtworzyć pierwotny kształt drzewa przy ponownym wstawianiu wartości, o ile wartości są unikalne i używany jest ten sam porządek porównań.

Żeby temu zapobiec, przy zapisie drzewa do pliku trzeba koniecznie użyć przechodzenia Preorder (Korzeń, Lewo, Prawo). Jeśli najpierw zapiszemy korzeń, a potem schodzimy w dół, to przy odczycie korzeń odtworzy się jako pierwszy. Każdy kolejny odczytany element naturalnie wpadnie na swoje stare, dobre miejsce, co pozwala idealnie zrekonstruować drzewo.

2.4. Zapis tekstowy a prawdziwa serializacja binarna

Kiedy zapisujemy dane w zwykłym pliku tekstowym, każda liczba jest zamieniana na znaki (ASCII). Problem w tym, że np. liczba "1024"zajmuje wtedy 4 bajty w

pliku, do tego musimy dodawać spacje, żeby program wiedział, gdzie kończy się jedna liczba, a zaczyna druga. Kiedy wczytujemy taki plik, komputer musi ciągle wyciągać te teksty i rzutować je z powrotem na liczby całkowite, co po prostu obciąża procesor.

W projekcie zastosowano zapis binarny z użyciem flagi `std::ios::binary`. Menedżer plików zapisuje kolejne wartości typu `int` za pomocą `file.write()` i `reinterpret_cast`. Rozmiar zapisu pojedynczej wartości wynosi `sizeof(int)` bajtów; nie można więc zakładać, że na każdej platformie będą to zawsze 4 bajty. Plik zawiera binarną reprezentację liczb, dlatego jego zawartość nie jest przeznaczona do czytania w zwykłym edytorze tekstu. Taki sposób zapisu nie zapisuje całych węzłów ani adresów wskaźników — przechowuje wyłącznie wartości, z których drzewo jest później odbudowywane.

Cecha	**Zapis Tekstowy (.txt)**	**Zapis Binarny (.bin)**
Format Danych textttint	Zwykły tekst (ASCII)	Binarna reprezentacja kolejnych wartości
Rozmiar w pliku	Zmienny (zależy od długości liczby + spacje)	Stały (zawsze 4 bajty dla zmiennej `int`)
Szybkość odczytu/zapisu	Niska (komputer musi konwertować string na int)	Bez konwersji tekstu na liczbę
Czytelność	Można łatwo podejrzeć i edytować w Notatniku	Odczyt tylko maszynowy (w Notatniku widać "krzaki")

Tab. 2.2. Zestawienie zapisu tekstowego i prawdziwego formatu binarnego.

2.5. Zarządzanie pamięcią i problem przepełnienia stosu

Ponieważ korzystamy z C++, nie mamy luksusu w postaci Garbage Collectora (jak w Javie), więc o pamięć trzeba dbać samemu. Każdy nowy węzeł drzewa jest alokowany dynamicznie za pomocą `new`. Jeżeli nie usuniemy go ręcznie, po jakimś czasie doprowadzimy do wycieku pamięci, który może zapchać cały RAM.

Aby temu zapobiec, napisałem funkcję `EraseNode`, która czyści pamięć używając metody Postorder. Jest to jedyny logiczny sposób na zniszczenie drzewa – musimy wejść na sam dół gałęzi i najpierw usunąć liście (dzieci), a dopiero na końcu ich rodzica za pomocą `delete`. Gdybyśmy spróbowali to zrobić w odwrotnej kolejności, odcielibyśmy sobie dostęp do niższych węzłów, a wskaźniki zaczęłyby pokazywać

losowe miejsca w pamięci. Podczas tworzenia algorytmów opartych na rekurencji napotkałem też na fizyczne ograniczenia systemu. Kiedy testowałem drzewo i zrobiło się ono mocno skośne (przypominało długą linię w dół), głęboka rekurencja błyskawicznie zużyła całą pamięć stosu systemowego. Wywaliło to program, rzucając błąd `STATUS_STACK_OVERFLOW` (kod wyjątku `0xC00000FD`). To doświadczenie pokazało, jak krytyczne jest poprawne ustawianie wskaźników na `nullptr` przy tworzeniu węzłów, żeby rekurencja wiedziała, kiedy bezpiecznie przestać się wywoływać.

3. Projektowanie

3.1. Narzędzia, technologie i ustawienia kompilatora

Architektura projektu została oparta na czystym języku C++, z naciskiem na ręczne, dynamiczne zarządzanie pamięcią. Jako główne środowisko programistyczne (IDE) wykorzystano CLion od JetBrains. Środowisko to znacząco usprawnia proces deweloperski, w szczególności dzięki wbudowanemu narzędziu do debugowania, które okazuje się niezastąpione podczas monitorowania wskaźników i zapobiegania błędom naruszenia ochrony pamięci (segmentation fault) przy operacjach na zdegenerowanych drzewach.

Za proces budowania aplikacji odpowiada system CMake. W pliku CMakeLists.txt zdefiniowano podstawowe flagi kompilatora oraz standard C++ (np. C++14), co pozwala zachować zgodność i przenośność na różnych systemach operacyjnych. Projekt nie wykorzystuje zewnętrznych, gotowych bibliotek do obsługi drzew ponieważ głównym założeniem było zaimplementowanie pełnej logiki struktury danych od podstaw.

Wykorzystano wyłącznie standardowe biblioteki języka C++: `<iostream>` do obsługi interfejsu konsolowego, `<vector>` do dynamicznego rejestrowania przebytej ścieżki podczas wyszukiwania węzłów oraz `<fstream>`. Ostatnia z wymienionych bibliotek odgrywa kluczową rolę w module menedżera plików, umożliwiając otwieranie strumieni z flagą `std::ios::binary`, co jest niezbędne do bezpośredniego zapisu surowych bajtów pamięci na dysk twardy.

3.2. Architektura systemu i diagram klas

Aby uniknąć nieczytelnego i monolitycznego kodu, projekt został podzielony na logiczne moduły zgodnie z zasadami programowania zorientowanego obiektowo. Fundamentalnym elementem architektury jest struktura Node, pełniąca rolę pojedynczego węzła. Przechowuje ona wartość całkowitą data oraz dwa wskaźniki na potomków (lewe i prawe dziecko). Struktura posiada dedykowany konstruktor, który automatycznie inicjalizuje wskaźniki wartością nullptr, co skutecznie zapobiega błędom związanym z odwoływaniem się do nieprzydzielonych obszarów pamięci.

Na tym fundamencie opiera się klasa `BST_System`, która hermetyzuje całą logikę matematyczną binarnego drzewa wyszukiwania – rekurencyjne wstawianie, usuwanie węzłów (wraz z procedurą poszukiwania następnika) oraz algorytmy prze-

chodzenia drzewa. Operacje dyskowe zostały wyodrębnione do niezależnej klasy `FileManager_System`, która przyjmuje wskaźnik na korzeń drzewa. Taki podział gwarantuje zachowanie zasady pojedynczej odpowiedzialności (Single Responsibility Principle) – logika drzewa jest całkowicie odizolowana od mechanizmów I/O, a menedżer plików operuje niezależnie od wewnętrznego mechanizmu sortowania węzłów.

Diagram UML klas w projekcie:

struct Node
+ data : int + L : Node* + R : Node*
+ Node() + Node(int val)

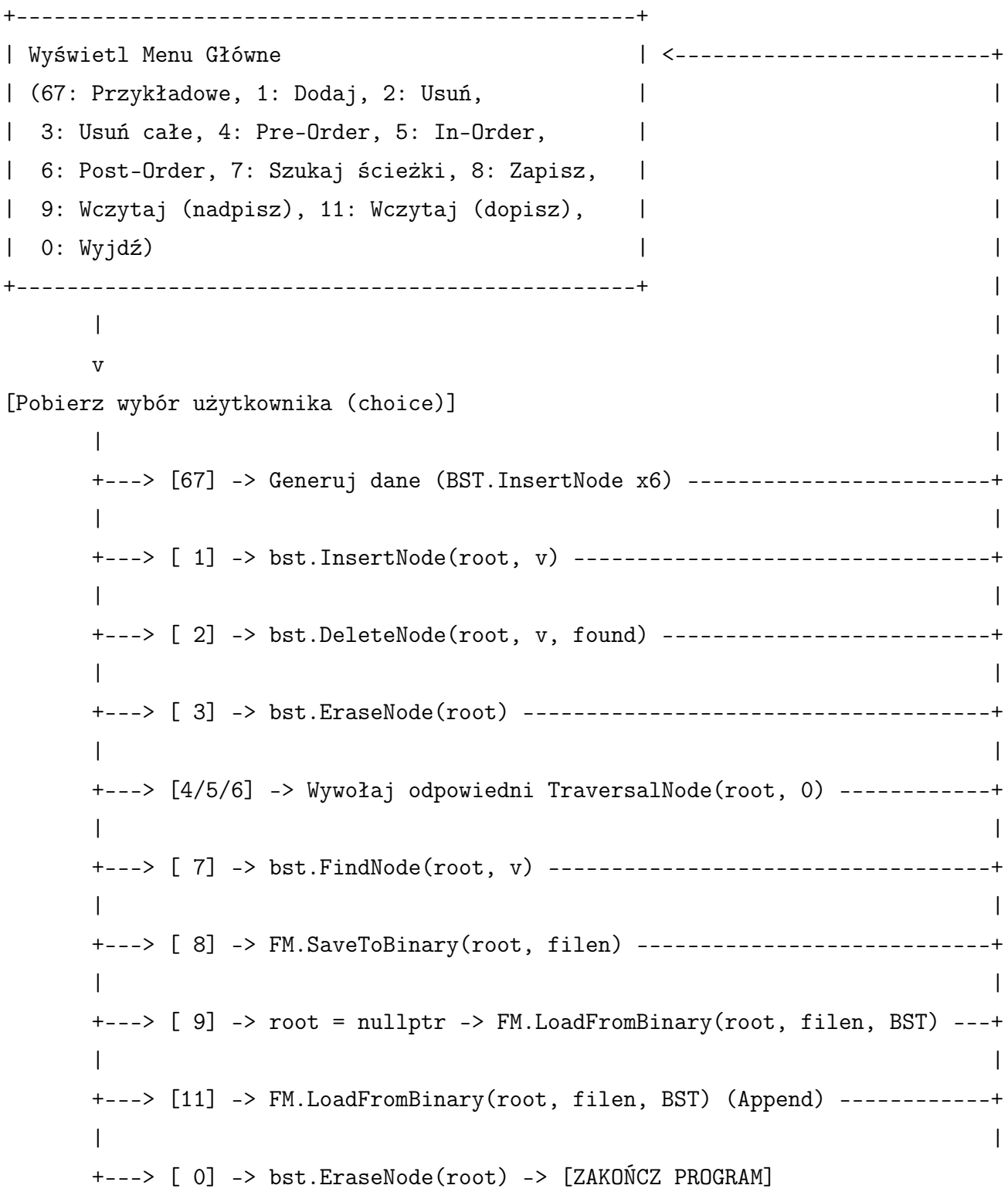
class BST_System
- data : int - L : Node* - R : Node*
+ BST_System(int o) + CreateNode(int d) : Node* + InsertNode(Node* root, int d) : Node* + DeleteNode(Node* root, int d, bool found) : Node* + FindNode(Node* root, int key) : Node* + FindPathRec(Node* node, int key, vector<int>& path) : bool + minValueNode(Node* node) : Node* + PreorderTraversalNode(Node* root, int depth) : void + InorderTraversalNode(Node* root, int depth) : void + PostorderTraversalNode(Node* root, int depth) : void + EraseNode(Node* node) : void

class FileManager_System (metody statyczne)
+ SaveToBinary(Node* root, const std::string& filename) : void + LoadFromBinary(Node* root, const std::string& filename, BST_System& bst) : Node* - SaveBinaryHelper(Node* root, std::ofstream& file) : void

Interfejsem integrującym poszczególne moduły jest nieskończona pętla `while` wyposażona w instrukcję `switch`, zaimplementowana w funkcji `main()`, która ob-

sługuje tekstowe menu użytkownika. Architektura głównego algorytmu sterującego aplikacją została zilustrowana na poniższym schemacie blokowym.

Schemat działania głównej pętli (Menu programu):



3.3. Zarządzanie kodem (Git) i generowanie dokumentacji

Chociaż omówiono już platformy GitHub i Doxygen, poniżej zamieszczono uszczegółowienie ich ról w procesie projektowym.

Niezwykle ważnym elementem nowoczesnego workflow jest zarządzanie historią kodu za pomocą systemu Git. Stanowi to doskonałe zabezpieczenie podczas pracy nad projektem – każdą nowo napisaną, działającą funkcję lub naprawiony błąd można zapisać jako commit, tworząc swego rodzaju punkt kontrolny. Wypychanie całości na zdalne repozytorium GitHub nie tylko tworzy bezpieczną kopię zapasową w chmurze, ale przede wszystkim znacząco usprawnia proces deweloperski. GitHub pozwala na precyzyjne śledzenie zmian, co okazuje się ratunkiem w sytuacjach, gdy dodanie nowej funkcjonalności negatywnie wpływa na stabilność całego drzewa BST. Aby utrzymać porządek w repozytorium i nie wysyłać do chmury niepotrzebnych artefaktów, kluczowe jest odpowiednie skonfigurowanie pliku `.gitignore`. Plik ten automatycznie blokuje wysyłanie skompilowanych plików binarnych (takich jak pliki `.exe`), obszernych folderów kompilacji (np. `cmake-build-debug`) oraz lokalnych konfiguracji środowiska IDE. Dzięki temu repozytorium pozostaje lekkie, przejrzyste i zawiera wyłącznie te pliki źródłowe, które są niezbędne do zbudowania i rozwijania programu na dowolnej innej stacji roboczej.

W kontekście tworzenia dokumentacji technicznej, optymalnym rozwiązaniem jest automatyzacja procesu zamiast ręcznego opisywania wszystkich klas i metod. W plikach nagłówkowych (`.h`) stosuje się specjalne formatowanie komentarzy kompatybilne z narzędziem Doxygen. Wykorzystanie dedykowanych tagów, takich jak `@brief` do ogólnego opisu, `@param` do zdefiniowania argumentów funkcji czy `@return`, pozwala na precyzyjne zmapowanie struktury kodu. Po zakończeniu implementacji uruchamia się interfejs graficzny Doxywizard, który analizuje pliki źródłowe. Zaznaczenie flagi `GENERATE_LATEX` sprawia, że Doxygen automatycznie ekstrahuje informacje z komentarzy i generuje kompletną, gotową strukturę dokumentacji w formacie plików `.tex`. Mechanizm ten eliminuje powtarzalną pracę i gwarantuje pełną zgodność dokumentacji z faktycznym stanem kodu źródłowego.

Ostatnim etapem jest złożenie wygenerowanych materiałów w jeden profesjonalny dokument, do czego idealnie nadaje się platforma Overleaf. Przeniesienie wygenerowanych przez Doxygena kodów źródłowych oraz tekstu sprawozdania do chmury Overleaf pozwala na bezproblemową kompilację ostatecznego pliku PDF. Zaletą tego rozwiązania jest przeniesienie całego procesu kompilacji na zewnętrzne serwery, co

całkowicie eliminuje konieczność instalowania i konfigurowania na stacji roboczej lokalnych dystrybucji LaTeXa (takich jak MiKTeX). Pozwala to uniknąć problemów z brakującymi pakietami i konfiguracją środowiska, umożliwiając skupienie się wyłącznie na dopracowaniu merytorycznym i wizualnym ostatecznego raportu akademickiego.

4. Implementacja

4.1. Szczegóły implementacyjne i wybrane fragmenty kodu

Proces implementacji oparto na bezpośrednim operowaniu wskaźnikami oraz dynamicznej alokacji pamięci na stercie. Zamiast korzystać z abstrakcyjnych kontenerów, struktura drzewa została zbudowana od podstaw. Poniżej przedstawiono analizę najciekawszych mechanizmów algorytmicznych zastosowanych w kodzie źródłowym.

Rejestrowanie ścieżki nawigacyjnej (`FindPathRec`)
Funkcja `FindPathRec` wyszukuje wskazany klucz i zapisuje w wektorze wartości kolejnych odwiedzonych węzłów, począwszy od korzenia. Wektor `std::vector<int>` jest przekazywany przez referencję, dzięki czemu kolejne wywołania rekurencyjne modyfikują tę samą ścieżkę. Funkcja zwraca `true`, jeśli klucz został znaleziony, albo `false` w przeciwnym przypadku.

```
bool BST_System::FindPathRec(Node* node, int key, std::vector<int>& path) {
    if (node == nullptr) return false;

    path.push_back(node->data); // Zapisanie biezacego wezla

    if (node->data == key) return true;
    if (key < node->data) return FindPathRec(node->L, key, path);

    return FindPathRec(node->R, key, path);
}
```

Powyższy fragment kodu ukazuje optymalizację procesu wyszukiwania. Elementy są odkładane na stos wektora w czasie rzeczywistym (`push_back`). Jeśli poszukiwania zakończą się sukcesem, rekurencja jest przerywana, a wywołująca funkcja `FindNode` otrzymuje gotową tablicę wartości do sformatowania i wyświetlenia w konsoli.

Wizualizacja hierarchii drzewa w konsoli (`TraversalNode`)

Zamiast tradycyjnego wypisywania wartości węzłów w jednej linii, zaimplementowano metody przechodzenia (np. `InorderTraversalNode`) w taki sposób, aby generowały dwuwymiarowy rzut struktury drzewa bezpośrednio w konsoli tekstowej. Osiągnięto to dzięki zastosowaniu parametru `depth`, który śledzi aktualny poziom zagłębienia rekurencji.

```
1 void BST_System::InorderTraversalNode(Node* root, int depth) {
2     if (root != nullptr) {
3         InorderTraversalNode(root->R, depth + 1);
4         for (int i = 0; i < depth; i++) std::cout << "    ";
5         std::cout << "--- " << root->data << "\n";
6         InorderTraversalNode(root->L, depth + 1);
7     }
8 }
```

Funkcja najpierw przechodzi do prawego poddrzewa, następnie wypisuje bieżący węzeł z wcięciem zależnym od wartości `depth`, a na końcu przechodzi do lewego poddrzewa. Dzięki temu drzewo jest prezentowane w konsoli w układzie obróconym: korzeń znajduje się bliżej lewej strony, a kolejne poziomy są przesunięte w prawo. W tej implementacji metoda nazwana `InorderTraversalNode` wypisuje wartości malejąco.

Bezpieczne usuwanie węzła z dwojgiem dzieci

Najbardziej złożonym przypadkiem strukturalnym jest usunięcie węzła, który posiada pełne rozgałęzienie (lewe i prawe dziecko). Aby zachować matematyczny inwariant BST, zaimplementowano metodę poszukiwania następnika in-order (najmniejszego elementu w prawym poddrzewie) za pomocą funkcji `minValueNode`.

```
// Fragment funkcji DeleteNode (wezel z dwojgiem dzieci)
Node* temp = minValueNode(root->R);

// Nadpisanie wartosci zamiast manipulacji wskaznikami
root->data = temp->data;

// Rekurencyjne usuniecie zduplikowanego nastepnika
root->R = DeleteNode(root->R, temp->data, found);
```

Ten fragment pokazuje standardowy sposób usuwania węzła mającego dwoje dzieci. Najpierw wyszukiwany jest najmniejszy element prawego poddrzewa, czyli następnik w porządku rosnącym. Jego wartość jest kopiowana do usuwanego węzła, a następnie rekurencyjnie usuwany jest pierwotny węzeł następnika. W aktualnej wersji parametr `found` jest jednak przekazywany przez wartość, więc jego zmiana wewnątrz funkcji nie jest widoczna w `main()`. Aby flaga działała zgodnie z zamierzeniem, parametr powinien być przekazywany przez referencję, np. `bool& found`.

Wstawianie z zachowaniem spójności strukturalnej (`InsertNode`)

Proces wstawiania nowego węzła został zaprojektowany z wykorzystaniem paradygmatu przypisywania zwrotnego, co eliminuje konieczność przekazywania wskaźników przez referencję lub stosowania podwójnych wskaźników (`Node**`).

```
1 Node* BST_System::InsertNode(Node *root, int data){
2     if (root == nullptr) return CreateNode(data);
3     if (data < root->data) root->L = InsertNode(root->L, data)
    ;
4     else if (data > root->data) root->R = InsertNode(root->R,
    data);
5     return root;
6 }
```

Kluczowym elementem tego algorytmu są przypisania `root->L = InsertNode(...)` oraz `root->R = InsertNode(...)`. Kiedy rekurencja dotrze do wolnego miejsca (`nullptr`) i utworzy nowy węzeł, funkcja zwraca jego adres. Przypisanie zwrotne gwarantuje, że wskaźnik węzła nadrzędnego zostanie natychmiast i bezbłędnie zaktualizowany, łącząc nową gałąź z resztą struktury.

5. Całkowita i bezpieczna destrukcja drzewa (`EraseNode`)

Zarządzanie dynamiczną pamięcią wymaga implementacji bezpiecznej metody niszczącej. Funkcja czyszcząca opiera się na matematycznym przechodzeniu wariantu Post-Order.

```
void BST_System::EraseNode(Node* node) {
    if (node != nullptr) {
        EraseNode(node->L);
        EraseNode(node->R);
        delete node;
    }
}
```

Zastosowana kolejność odpowiada przejściu postorder: najpierw usuwane są lewe i prawe poddrzewo, a dopiero później bieżący węzeł. Dzięki temu funkcja nie odwołuje się do dzieci po zwolnieniu pamięci rodzica. Sama funkcja przyjmuje jednak wskaźnik przez wartość, więc nie ustawia wskaźnika korzenia w `main()` na `nullptr`. Po jej wywołaniu należy jawnie wykonać `root = nullptr`; w przeciwnym razie zmienna `root` będzie zawierała nieaktualny adres.

4.2. Powstałe wyniki działania programu

Program udostępnia opisane niżej funkcje. Należy jednak oddzielić zamierzone działanie algorytmów od ograniczeń obecnej implementacji: nie wszystkie zachowania można uznać za w pełni sprawdzone lub bezbłędne:

- **Wizualna weryfikacja poprawności:** Dzięki zmodyfikowanym algorytmom przechodzenia drzewa (z parametrem `depth`), wyrenderowane w konsoli wyniki jednoznacznie potwierdzają prawidłowe, hierarchiczne ułożenie węzłów. Wstawienie przykładowych wartości (np. 50, 60, 25) skutkuje precyzyjnym graficznym rozgałęzieniem, co pozwala użytkownikowi na natychmiastową weryfikację poprawności działania algorytmu wstawiania.
- **Śledzenie ścieżek nawigacyjnych:** Wyszukanie konkretnego elementu za pomocą zaimplementowanej opcji z menu skutkuje wygenerowaniem sformatowanego wyniku, np. `Droga do wartości 20: 50 -> 30 -> 20`. Pokazuje to, że podczas wyszukiwania sprawdzane są kolejne węzły na ścieżce prowadzącej do szukanego klucza.
- **Obsługa nieznalezionych wartości:** Funkcja wyszukiwania może rozpoznać brak szukanego klucza. W przypadku usuwania flaga `found` jest obecnie

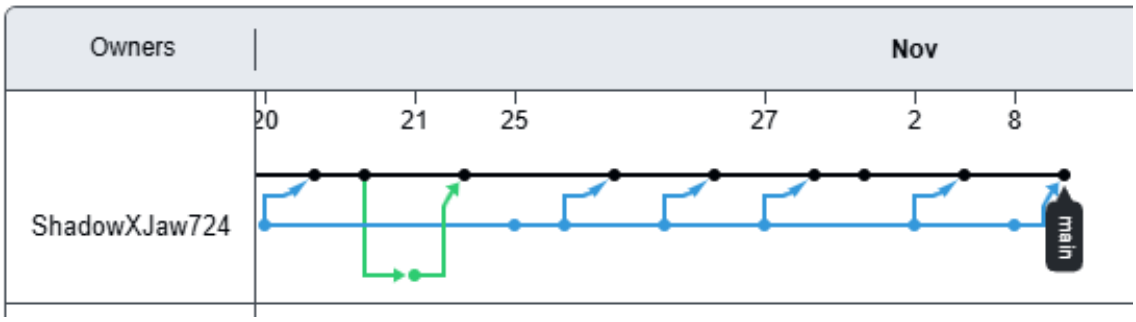
przekazywana przez wartość, a nie przez referencję, więc komunikat oparty na tej fladze może być nieprawidłowy. Nie należy deklarować pełnej odporności programu przed poprawieniem tego fragmentu.

- **Zwalnianie pamięci:** Funkcja `EraseNode` przechodzi po drzewie w kolejności postorder i zwalnia pamięć każdego węzła za pomocą `delete`. Ponieważ przyjmuje wskaźnik przez wartość, po jej wywołaniu należy dodatkowo ustawić wskaźnik korzenia na `nullptr`. Bez tego zmienna `root` nadal przechowuje nieaktualny adres.

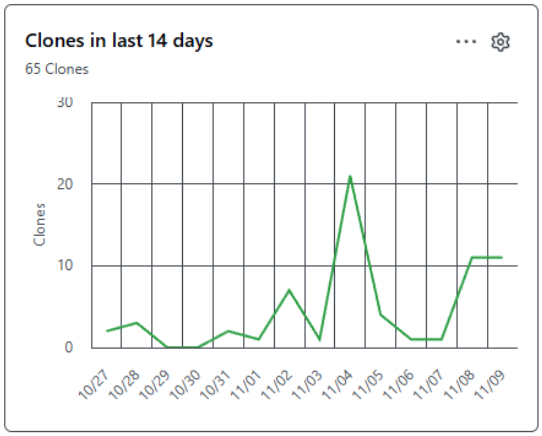
- **Zapis i odczyt danych:** Menedżer zapisuje wartości typu `int` w kolejności preorder. Przy odczycie każda wartość jest ponownie wstawiana do drzewa. Dla unikalnych wartości i zgodnego porządku porównań taka kolejność pozwala odtworzyć strukturę drzewa. Plik zawiera same wartości, a nie pełne węzły ani wskaźniki. Rozmiar zapisu pojedynczej wartości zależy od `sizeof(int)`.

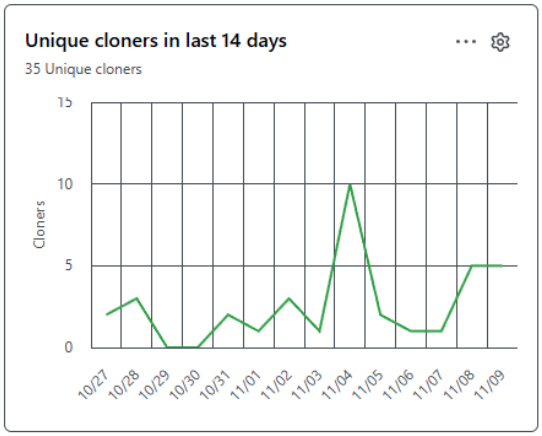
5. Wnioski

Projekt zaprezentował sposoby i zastosowania drzewa wyszukiwań binarnych oraz zarządzania plikami, a także sposób implementacji obu tych elementów w programie. Poniżej znajdują się statystyki GitHuba dla zainteresowanych.

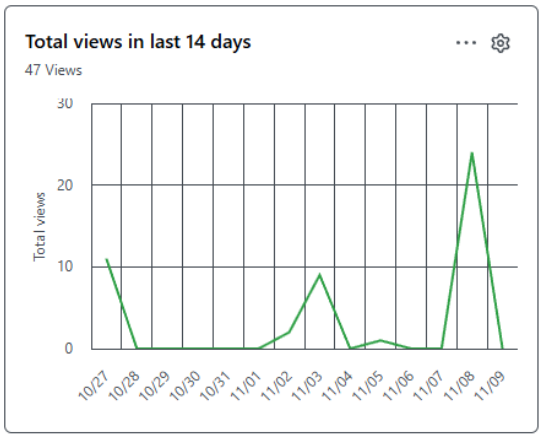


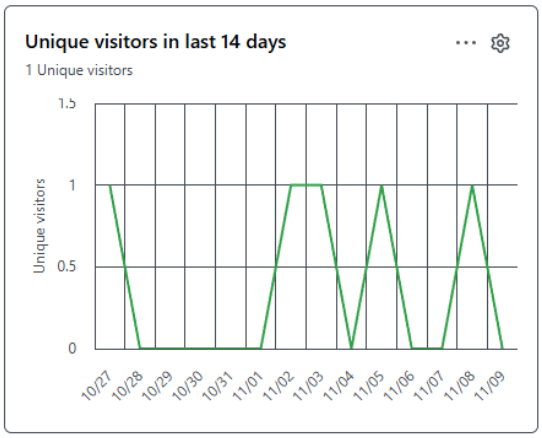
Git clones





Visitors





Spis rysunków

1.1. Binary Search Tree Wizualny Pokaz 3
1.2. Doxygen diagram jak dziala 8

Spis tabel

2.1. Złożoność operacji na drzewie BST. 12

2.2. Zestawienie zapisu tekstowego i prawdziwego formatu binarnego. . . . 13

Spis listingów